RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering



Object Oriented Methodologies (CS224)

By

Dr. Kalka Dubey

Object Oriented Methodologies (CS224)
11 (3-0-2)

Dr. Kalka Dubey Assistantt Professor RGIPT Jais Amethi

Syllabus

UNIT 1: Fundamental concepts of object oriented programming:

UNIT 2: Classes & Objects

UNIT 3: Inheritance & Polymorphism:

UNIT 4: Standard Template Library (STL) of C++

UNIT 5: Packages & Exception Handling

Objective:

To study the concept of object-oriented programming and practices of C++ programs that leverage the object-oriented methodology features.

Text and Referred Books

Text books:

*1. **Object Oriented Programming With C++ by E. Balagurusamy, McGraw-Hill Education (India) Pvt.***

2. Object-oriented programming in C++ by Lafore, R. Waite Group.

Reference books:

1. Functional and Object-Oriented Analysis and Design by Peretz Shoval, IGI Publishing.

2. Structured and Object-Oriented Analysis and Design Methodology by Pankaj Ghemawat, Pearson Education.

Outcome of the Course

The following are the official course goals agreed upon by the faculty for this course:

- *Concepts of classes & objects and their significance use in the real world.*
- *Benefits of object oriented design and methodology*
- *Build C++ classes using appropriate encapsulation and design principles.*
- *Analyse the utilization of inheritance and polymorphism in the solution of problems.*
- *Choose appropriate object orient programming concepts for solving real-world problems.*

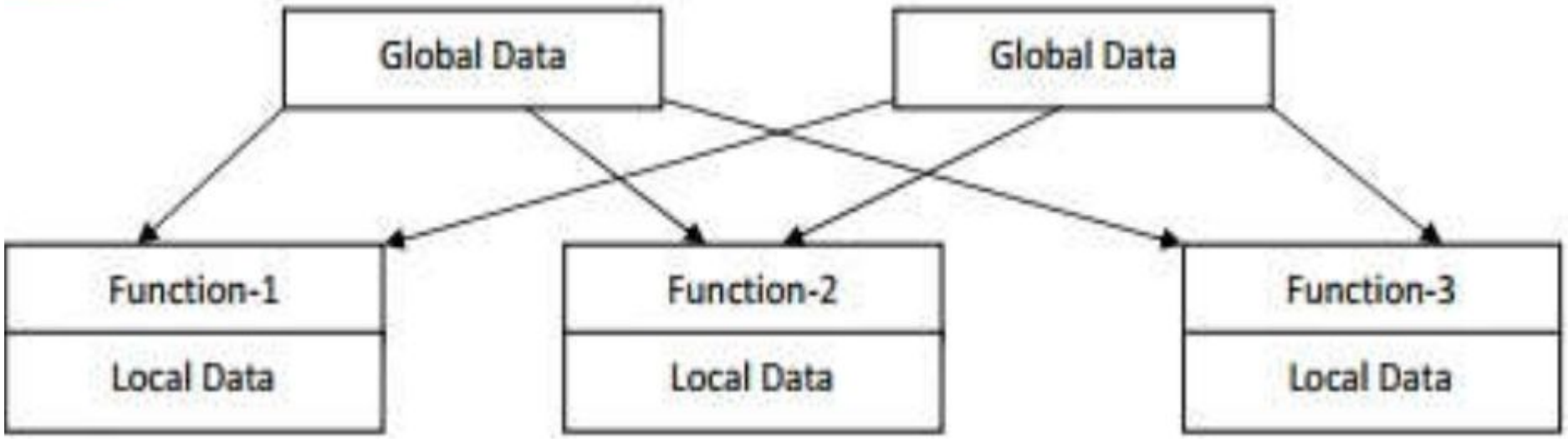
UNIT-I

- *Review of Structured Programming*
- *Structured versus Object Oriented Programming (OOP)*
- *Features of OOP*
- *Basics of C++*
- *Tokens, I/O*
- *Statements*
- *Structure of Program*
- *Operators, and Expressions*
- *Flow of Control*
- *Abstract Data Types (ADT)*
- *Variable*
- *Scope Resolution*
- *Operator*

Structured Programming

- Conventional programming, using **high level languages** such as COBOL, FORTRAN, and C, is commonly known as Structural programming .
- The problem is viewed as **a sequence of things** to be done such as reading, calculating, and printing.
- A number of **functions are written** to accomplish these tasks.
- Divide the whole program into **small modules**, so that the program becomes easy to understand.

Structured Programming



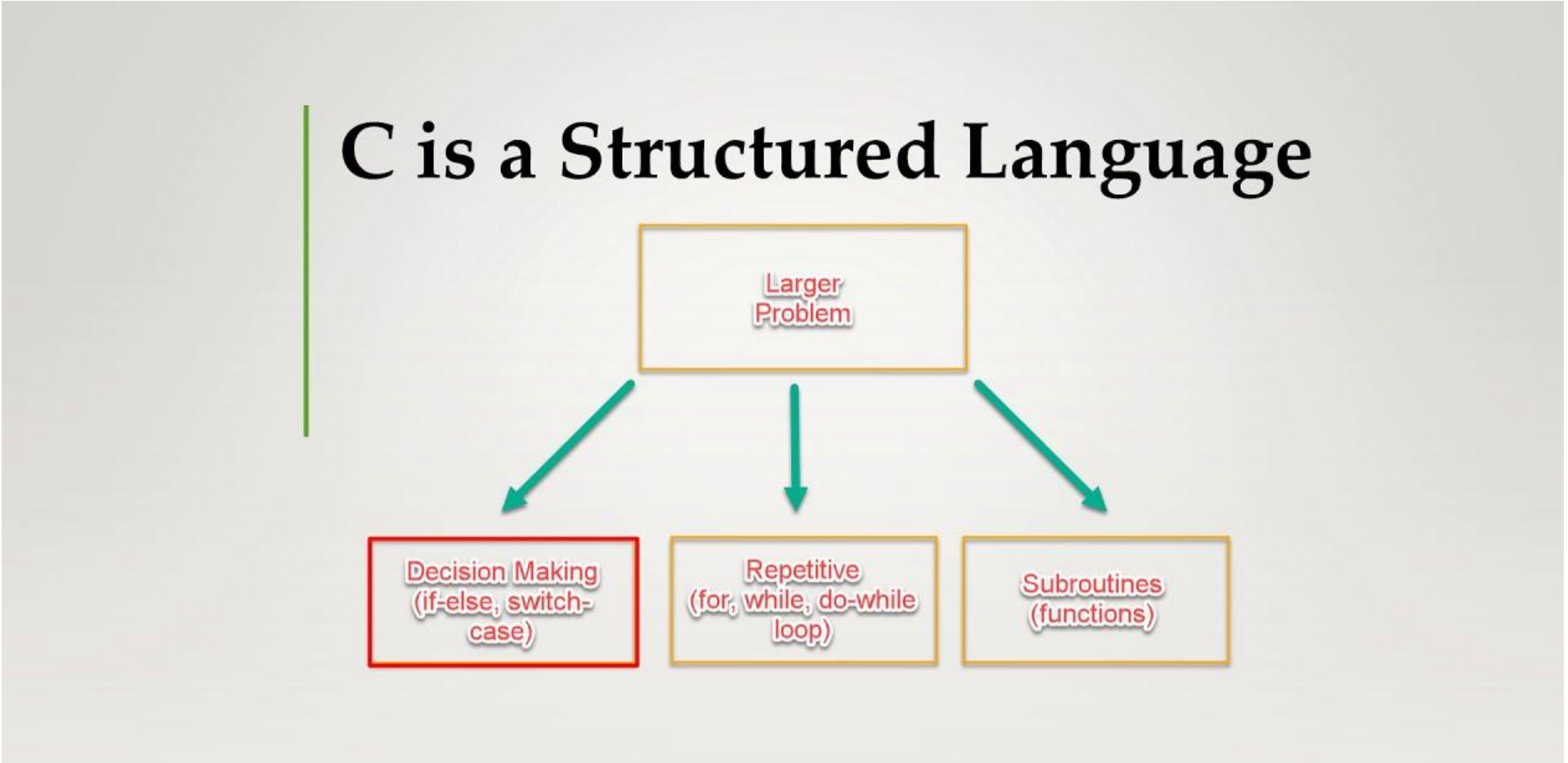
Structured Programming

- It enables the programmer to understand the **program easily**.
- Easy to **debug** the error.
- Time Efficient.

Features of Structured Programming

- Top Down Design.
- Code reusability.
- Information hiding.

Structured Programming



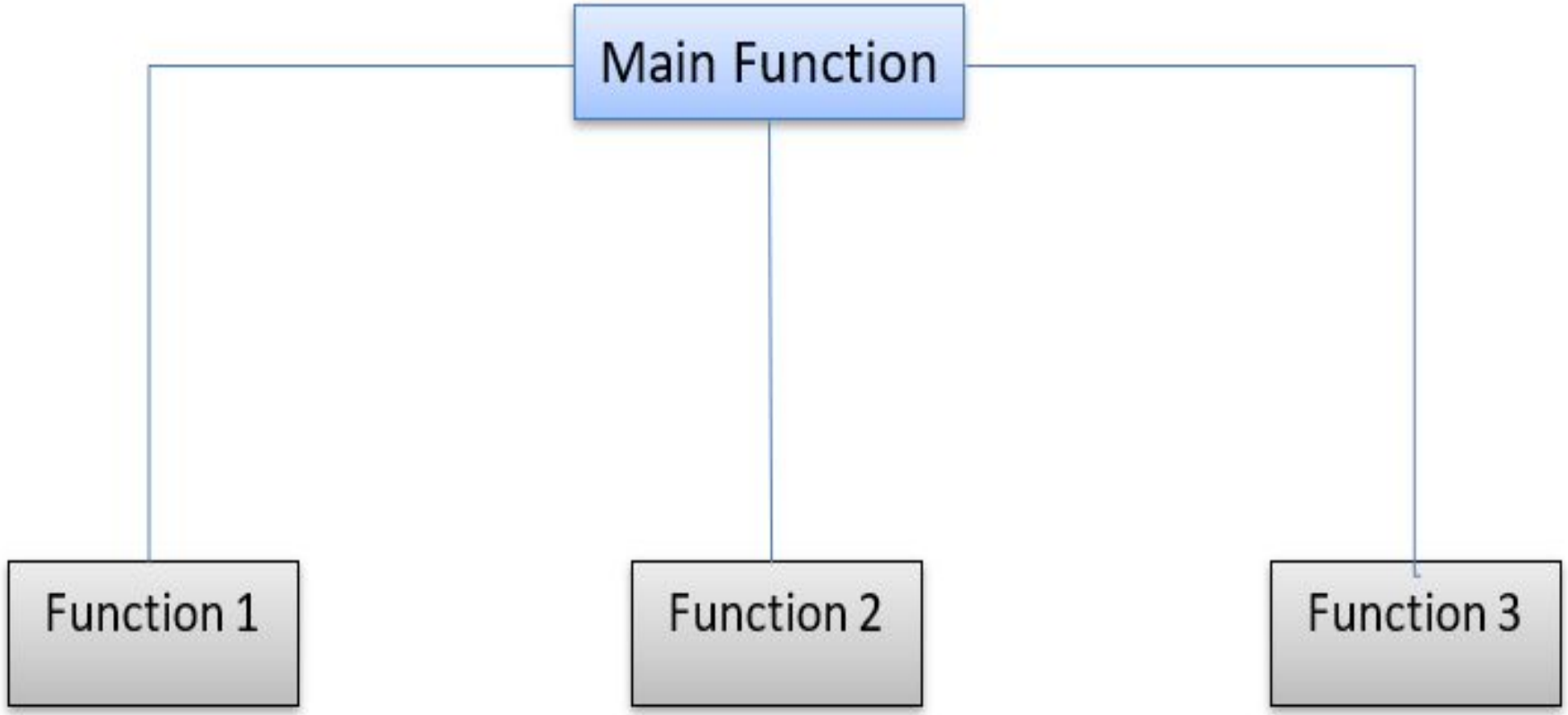
Function

- Structural Programming divide the whole program in to small blocks called functions.
- Function is a block of statements that are executed to perform a task.
- Function plays an important role in structured programming.
- Using a function is like hiring a person to do a specific job.

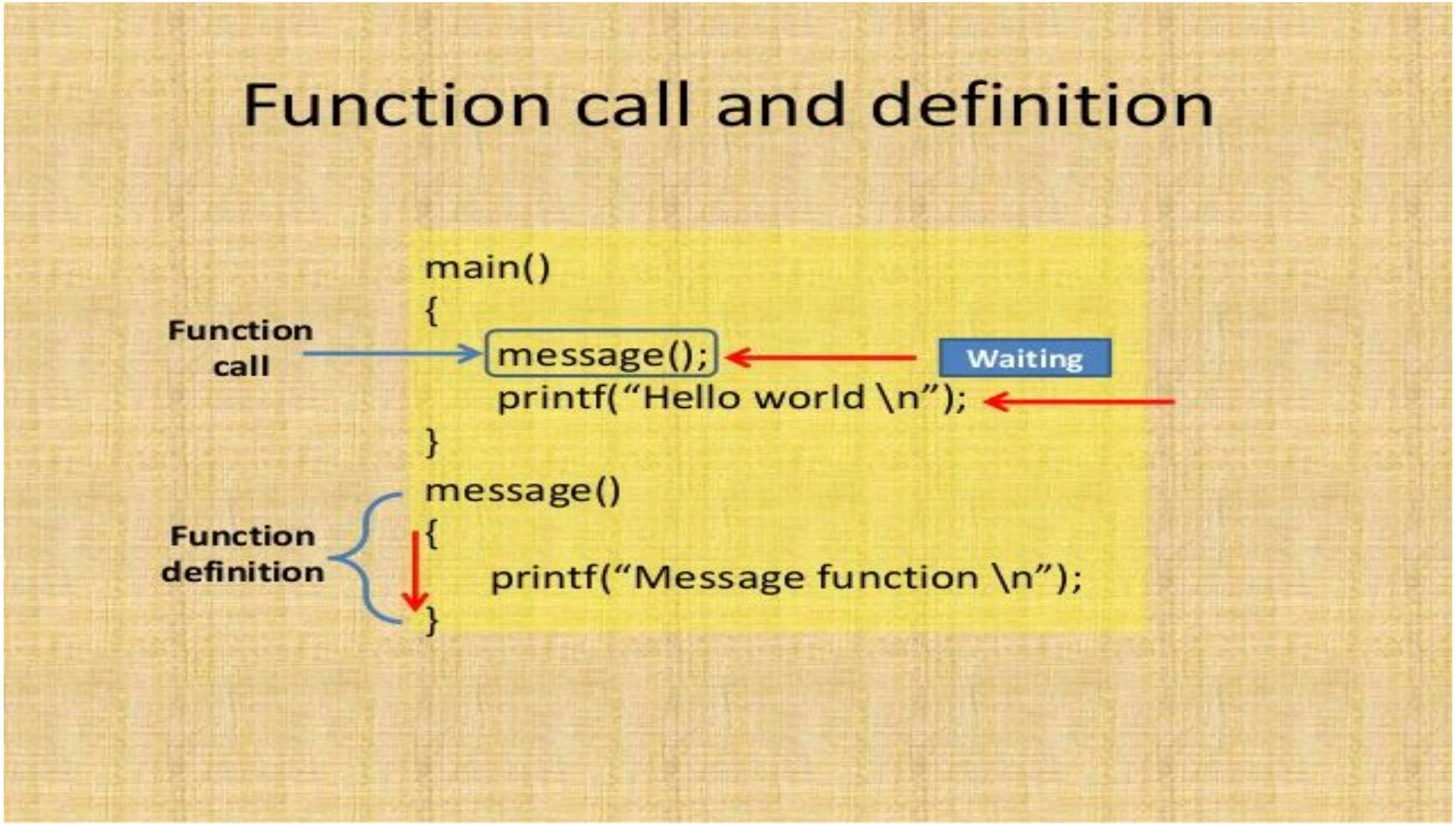
Function conti…

- When a function is called by its name the control moves to the function definition and executes all the statements written in it.
- Semi colon is used after the name of function.
- Function Definition : It contains the instructions that are executed when a function is called
- Semi colon is not used after the name of function.

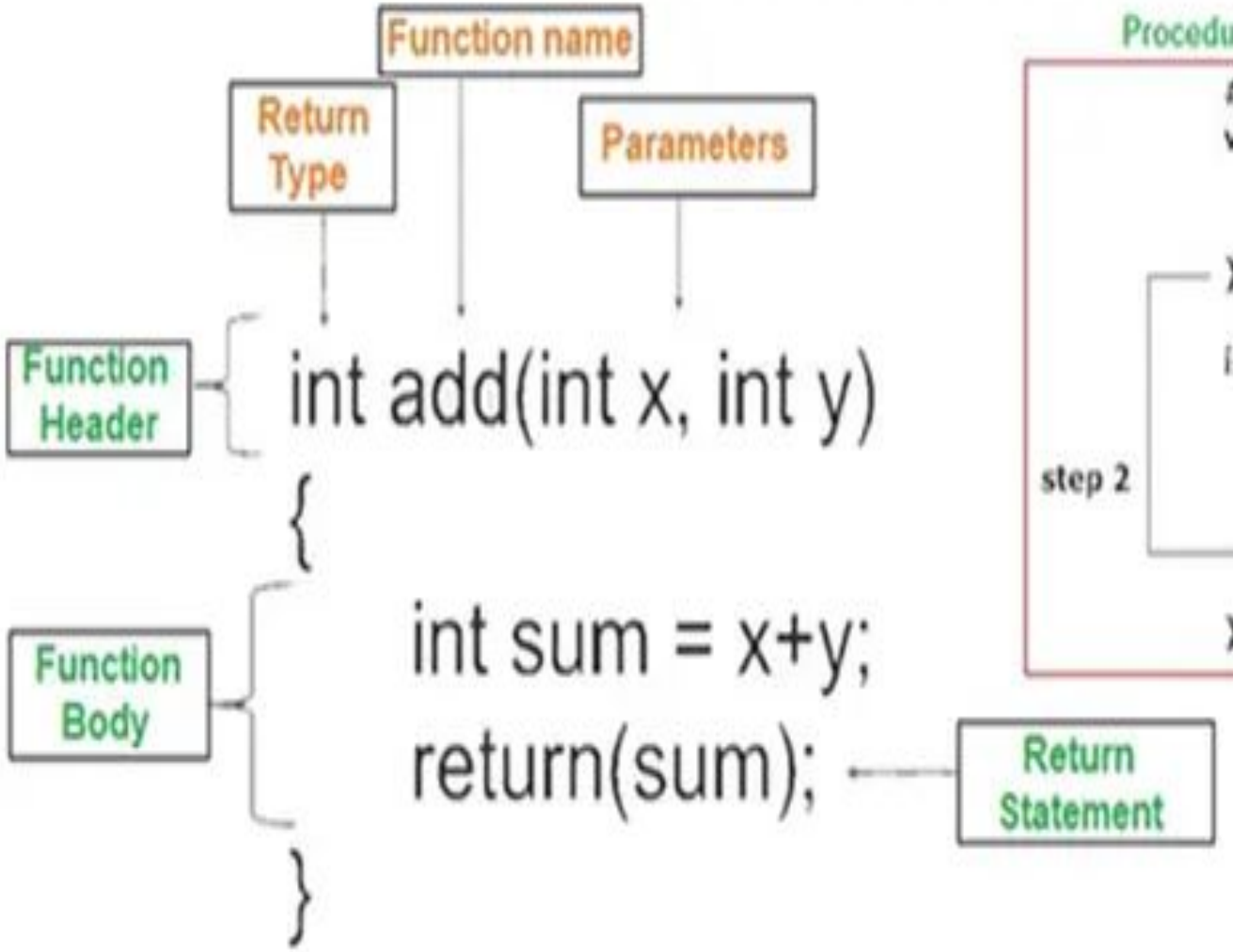
Function

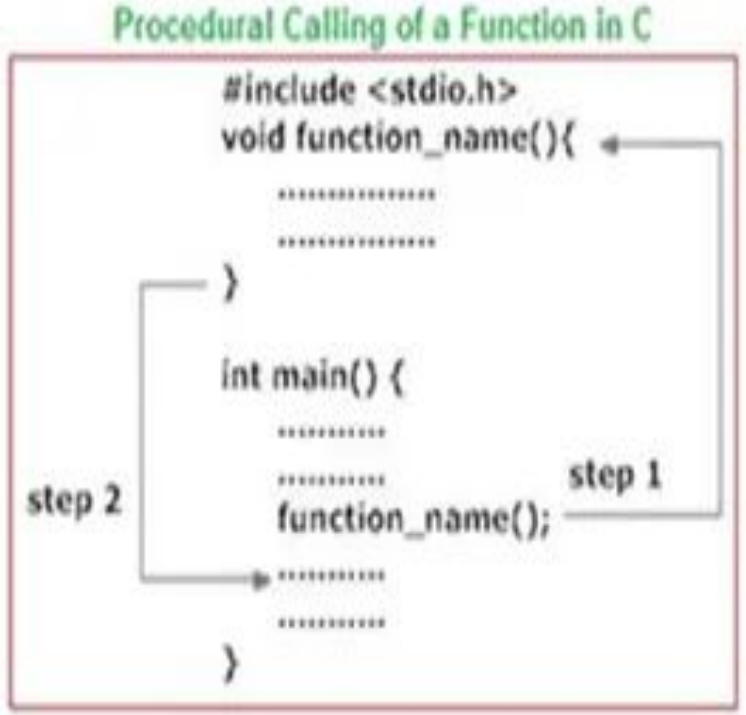


Function


Function call and definition
main()
{
message();
printf("Hello world \n");
}
message()
{
printf("Message function \n");
}
Function call
Waiting
Function definition

Function





Output ?

Object-Oriented Programming

- A rich style in program structure based on a collection of classes and object.
- Most popular current representatives are C++, and Java.
- An object-oriented design model.
- Principle programming techniques.
- Relation to other models (higher-order programming, component based programming, functional).

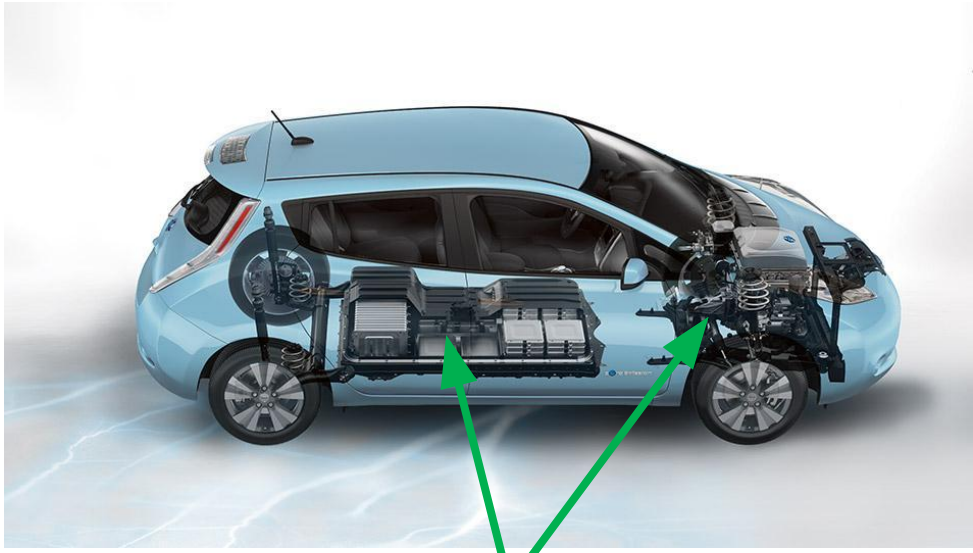
Object Oriented Programming

- OOP defines *classes* to represent these things.
- Classes can contain data and methods (internal functions).
- Classes control access to internal data and methods.
- A public interface is used by external code when using the class.
- This is a highly effective way of modeling real world problems inside of a computer program.

"Class Car"

public interface



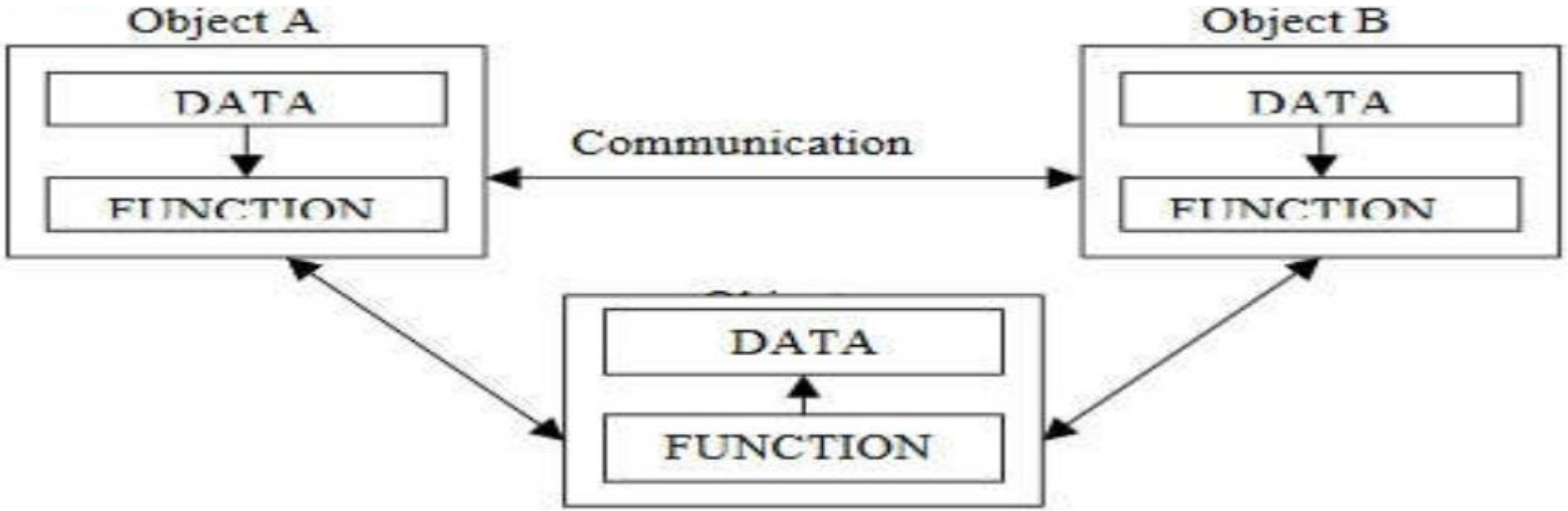


private data and methods

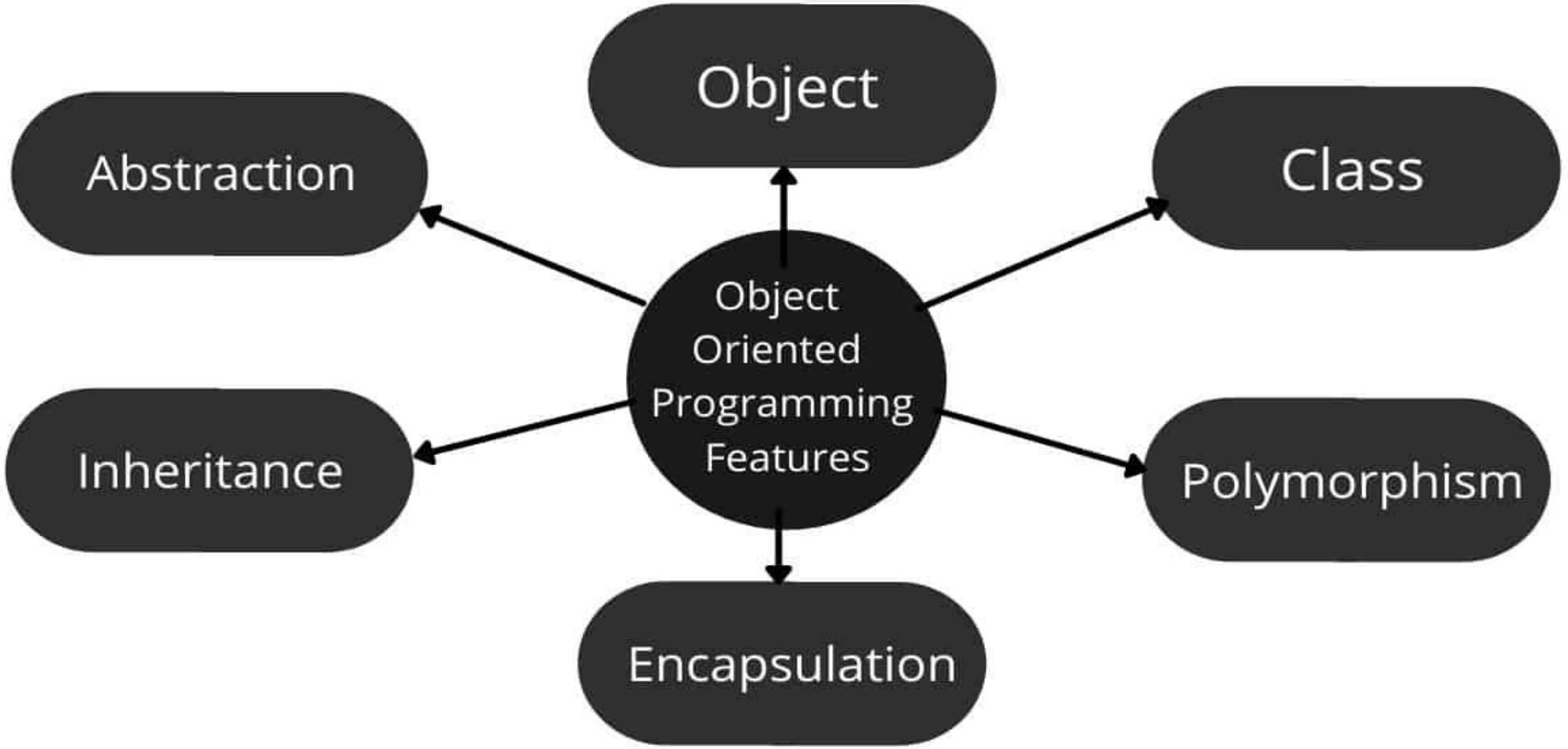
Object-Oriented Programming

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- Follows bottom-up approach in program design

Object Oriented Programming



Object Oriented Programming features



Object Oriented Programming features

- Object oriented programming concept use to make powerful, robust and secure programming instruction.
- Each program is made up of many entities called Classes.
- Objects become the fundamental units and have behaviour, or a specific purpose, associated with them.
- Objects cannot directly access another object's data.

Advantages of Object-oriented programming

- *Real world programming.*
- *Ability to simulate real-world event much more effectively*
- *Reusable of code.*
- *Information hiding.*
- *Data becomes active*
- *Better able to create GUI (graphical user interface) applications*
- *Programmers are able to reach their goals faster*
- *Programmers are able to produce faster, more accurate and better-written applications.*

Applications of Object-oriented programming

- *Real-business systems are often much more complex and contain many more objects with complicated attributes and methods.*

- *OOP is useful in these types of applications because it can simplify a complex problem. The promising areas for application of OOP include:*

- *Simulation and modelling*
- *Object-oriented databases*
- *Hypertext, hypermedia and expert Ext*
- *Al and expert systems*
- *Neural networks and parallel programming*
- *Decision support and office automation systems*
- *CIM/CAM/CAD systems*

Difference between Object Oriented Programming and Structural Programming

- *Structural programming creates a step by step program that guides the application through a sequence of instructions. While Object-Oriented programming is much more similar to the way the real world works.*
- *In Structural programming each program is made up of many small functions. While in Object-Oriented programming each program is made up of many entities called objects and class.*
- *Structural programming focuses on the idea that all algorithms are executed with functions and data that the programmer has access and Each instruction is executed in order. While in Objects become the fundamental units and have behavior, or a specific purpose, associated with them*

Difference between Object Oriented Programming and Structural Programming

S. No.	Structured Programming	Object-Oriented Programming
1	It is a subset of procedural programming.	It relies on concept of objects that contain data and code.
2	Programs are divided into small programs or functions.	Programs are divided into objects or entities.
3	It is all about facilitating creation of programs with readable code and reusable components.	It is all about creating objects that usually contain both functions and data.
4	Its main aim is to improve and increase quality, clarity, and development time of computer program.	Its main aim is to improve and increase both quality and productivity of system analysis and design.
5	It simply focuses on functions and processes that usually work on data.	It simply focuses on representing both structure and behaviour of information system into tiny or small modules that generally combines data and process both.
6	It is a method of organizing, managing and coding programs that can give or provide much easier modification and understanding.	It is a method in which set of objects can vary dynamically and can execute just by acting and reading to each other.

Difference between Object Oriented Programming and Structural Programming

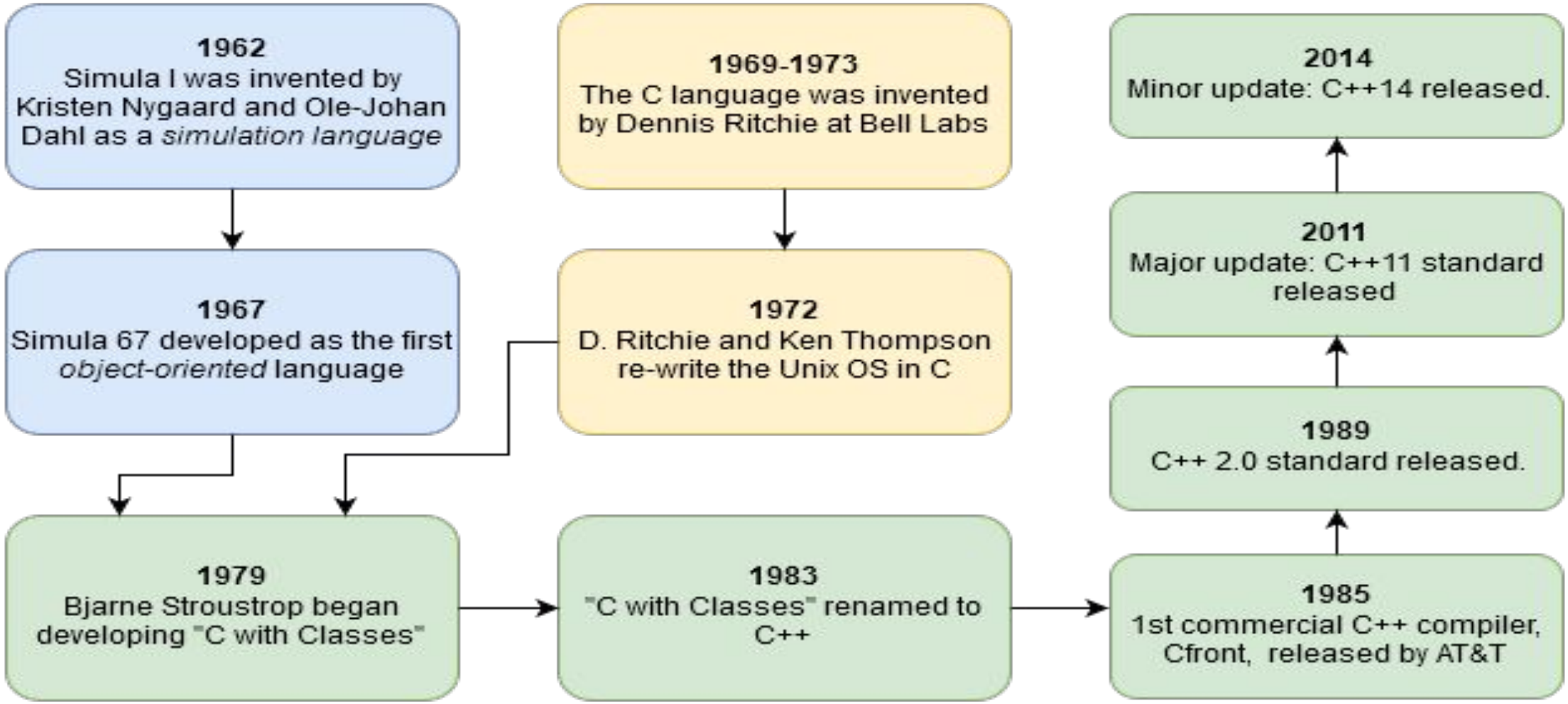
S. No.	Structured Programming	Object-Oriented Programming
7	In this, methods are written globally and code lines are processed one by one i.e., Run sequentially.	In this, method works dynamically, make calls as per need of code for certain time.
8	It generally follows “Top-Down Approach”.	It generally follows “Bottom-Up Approach”.
9	It provides less flexibility and abstraction as compared to object-oriented programming.	It provides more flexibility and abstraction as compared to structured programming.
10	It is more difficult to modify structured program and reuse code as compared to object-oriented programs.	It is less difficult to modify object-oriented programs and reuse code as compared to structured programs.
11	It gives more importance of code.	It gives more importance to data.
12	It gives less importance of security of data.	It gives less importance of security of data.

C++ Language

Exercises

1. Write a program in C++ to find the area of circle using input value ?
2. Write a program in C++ to Arithmetic operations of two integer numbers ?
3. Write a program in C++ to print the upper case letters to lower case ?
4. Write a program in C++ to show of a numeric value of a variable using hex, oct, and dec manipulator functions. ?
5. Write a program in C++ to check whether the given character upper, lower , numbr or not ?
6. Write a program in C++ to find biggest of three given numbers ?
7. Write a program in C++ to display the name of the day in a week .here given input range is 1 to 7. ?
8. Write a program in C++ to display the even or odd numbers between 1 to 100 using while loop ?
9. Write a program in C++ to print a word form of given number between 0 to 9 using Switch ?
10. Write a program in C++ to find the smallest of 10 numbers ?

History of C++


1962
Simula I was invented by Kristen Nygaard and Ole-Johan Dahl as a *simulation language*
1967
Simula 67 developed as the first *object-oriented* language
1969-1973
The C language was invented by Dennis Ritchie at Bell Labs
1972
D. Ritchie and Ken Thompson re-write the Unix OS in C
1979
Bjarne Stroustrop began developing "C with Classes"
1983
"C with Classes" renamed to C++
1985
1st commercial C++ compiler, Cfront, released by AT&T
1989
C++ 2.0 standard released.
2011
Major update: C++11 standard released
2014
Minor update: C++14 released.

Characteristics of C++

"Actually I made up the term 'object-oriented', and I can tell you I did not have C++ in mind."
– Alan Kay (helped invent OO programming, the Smalltalk language, and the GUI)

- C++ is a
  - Compiled.
    - A separate program, the compiler, is used to turn C++ source code into a form directly executed by the CPU.
  - Strongly typed and unsafe
    - Conversions between variable types must be made by the programmer (strong typing) but can be circumvented when needed (unsafe)
  - C compatible
    - call C libraries directly and C code is nearly 100% valid C++ code.
  - Capable of very high performance
    - The programmer has a very large amount of control over the program execution
  - Object oriented
    - With support for many programming styles (procedural, functional, etc.)
- No automatic memory management
  - The programmer is in control of memory usage

Difference b/w c and c++

C Language	C++ Language
Procedural Programming Language	Object Oriented Programming Language
Headerfile : #include<stdio.h>	#include<iostream.h>
"\n" is used to go to the next line	we can use endl statement
Local variables declared only the start of a C program	It can be declared anywhere in a program, before they are used
Return type for a function is optional	Return type must be specified
Do not permit data Hiding	They permit data hiding
Bydefault structure members are public	class members are private
We can call a main() function within a program	This is not allowed

BASIC TERMINOLOGY of C++

object - An object is an instance of a class which combines both data and functions together.

Method - an action performed by an object (a verb)

Attribute - description of objects in a class

class - a category of similar objects

- does not hold any values of the object's attributes

By default all members declared inside a class are private to that class

Basic of C++

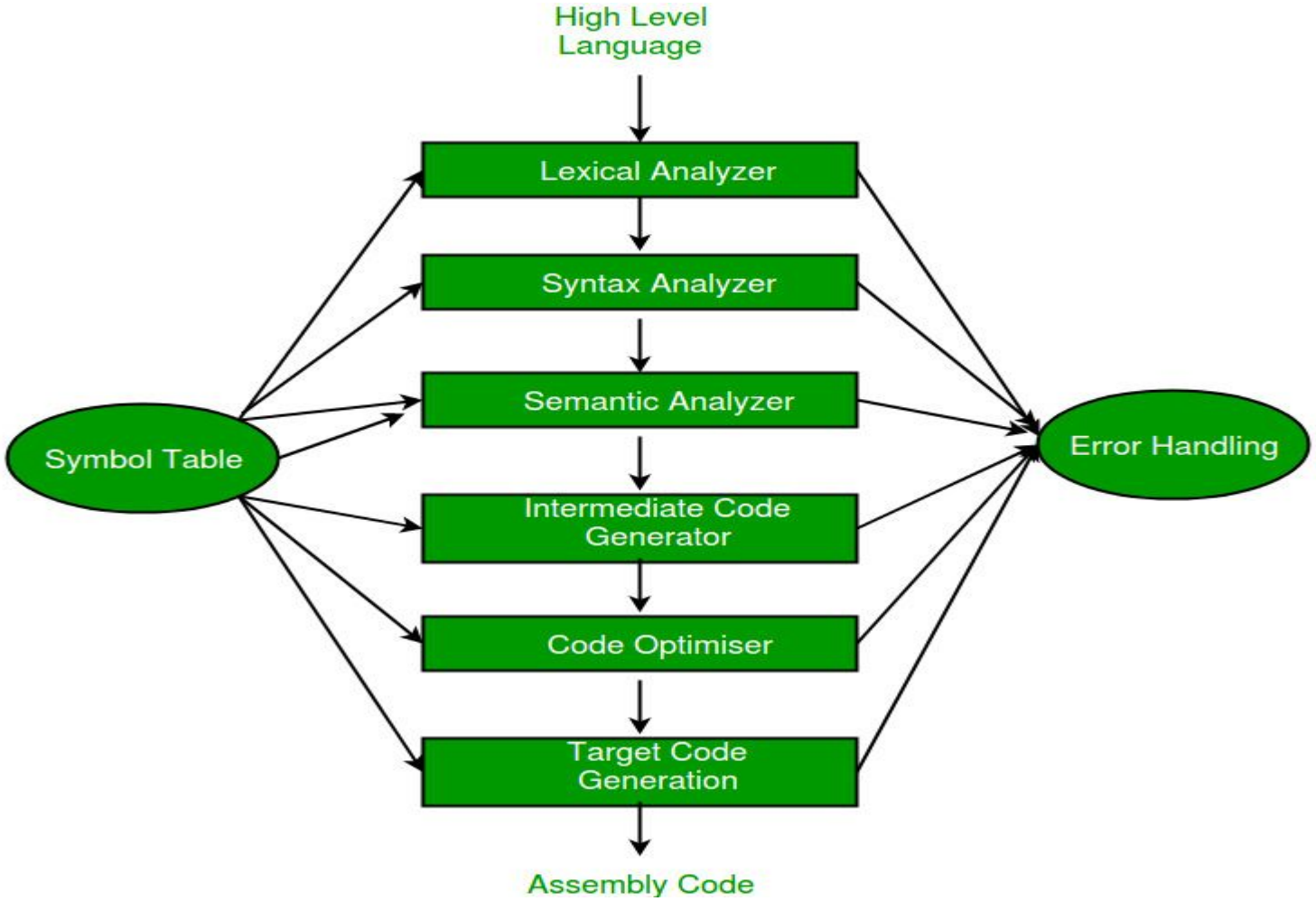
1. Compilation
2. The # Preprocessor
3. Declarations and Definitions
4. main () function
5. Comments
6. Class Variables & Functions
7. Input, output operators
8. Cascading I/O operators
9. Control Statement

1. Compilation

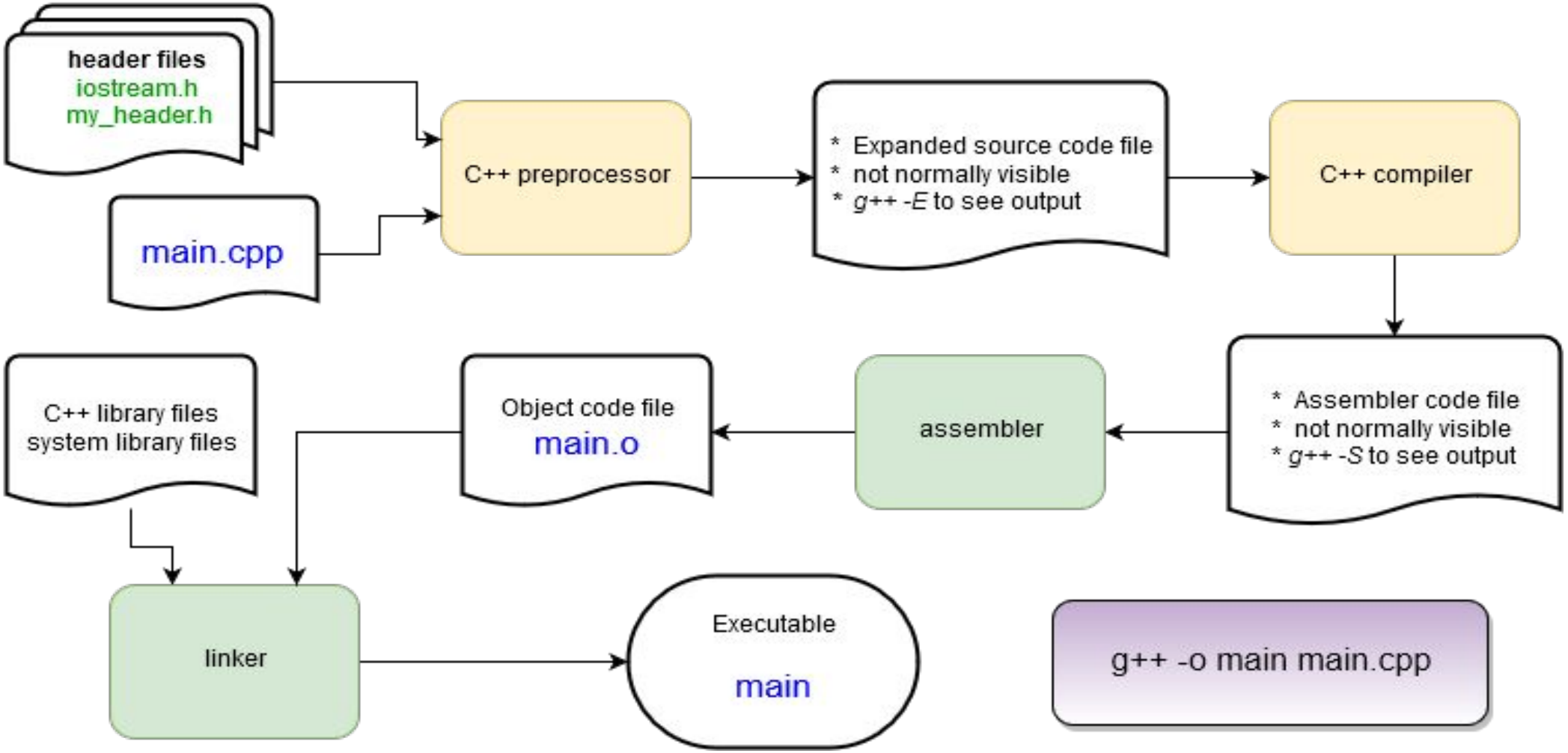
- C++ programs can range from a handful of statements to hundreds of thousands.

- May be written by one person or by a team

1. Compilation



Behind the Scenes: The Compilation Process

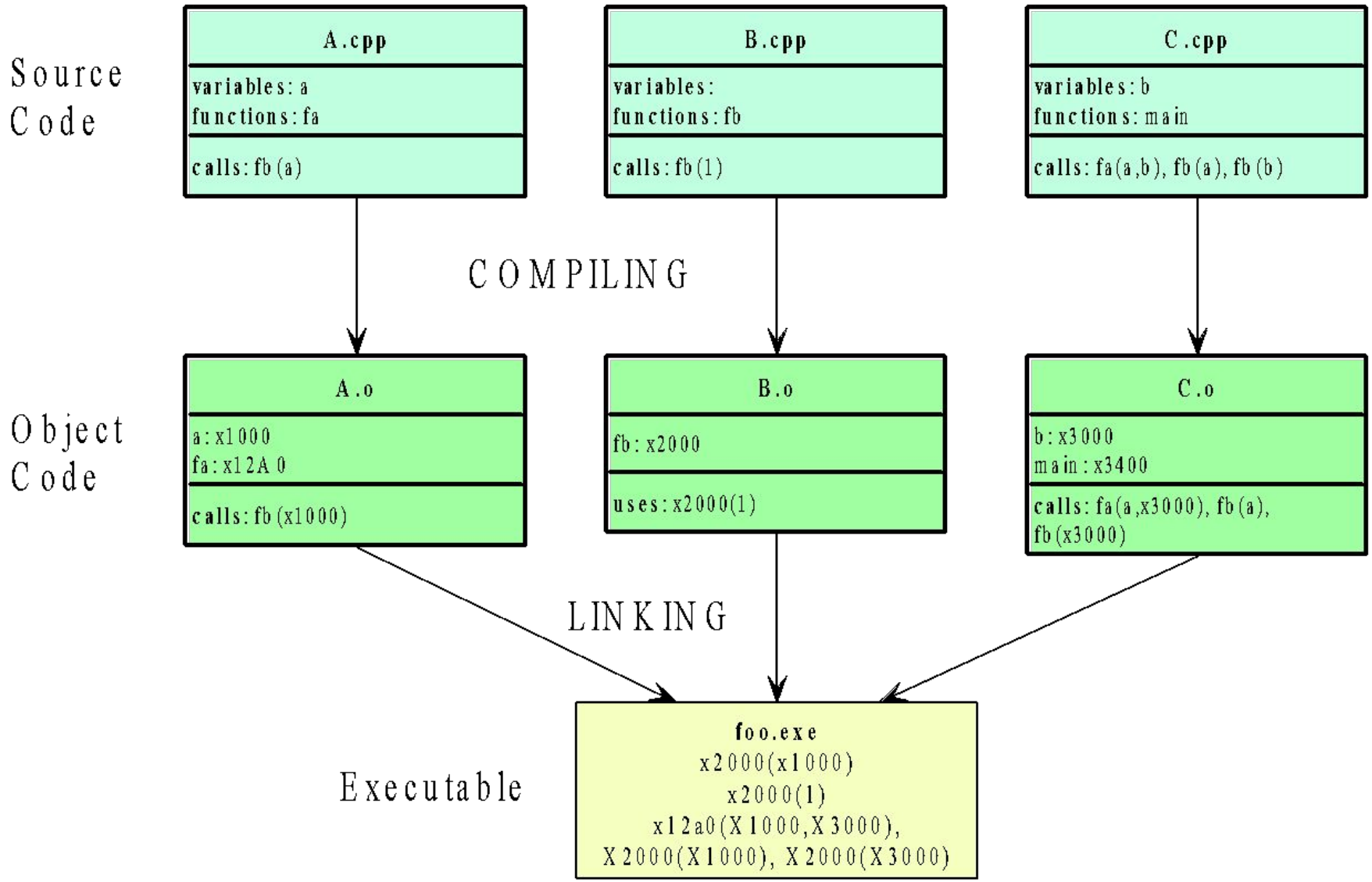


Multiple File C++ Programs

- Team members can work in parallel on separate files.
- Files are compiled separately
  - each individual compilation is fast
- Separately compiled code is linked to produce the executable
  - linking is much faster than compilation

Separate Compilation

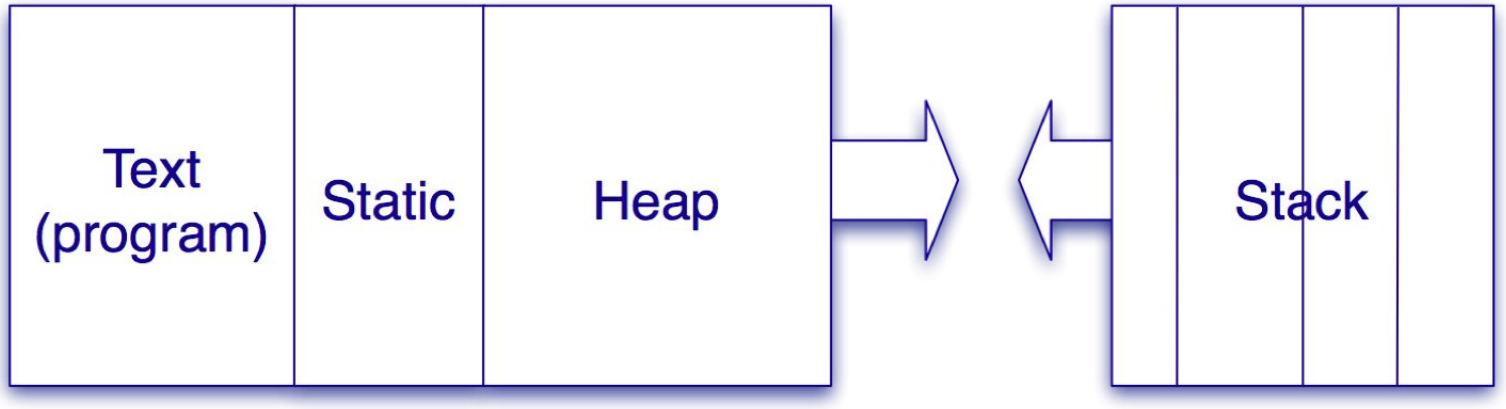
- Each file of source code
  - programming language text
- is compiled to produce a file of object code
  - binary code, almost executable
  - exact addresses of variables and functions not known, represented by symbols
- All object code files linked to produce the executable
  - Linking mainly consists of replacing symbols by real addresses.


Source Code
A.cpp
variables: a
functions: fa
calls: fb(a)
B.cpp
variables:
functions: fb
calls: fb(1)
C.cpp
variables: b
functions: main
calls: fa(a,b), fb(a), fb(b)
COMPILING
Object Code
A.o
a: x1000
fa: x12A0
calls: fb(x1000)
B.o
fb: x2000
uses: x2000(1)
C.o
b: x3000
main: x3400
calls: fa(a,x3000), fb(a), fb(x3000)
LINKING
Executable
foo.exe
x2000(x1000)
x2000(1)
x12a0(X1000,X3000),
X2000(X1000), X2000(X3000)

Memory Layout

The address space consists of (at least):

Text:	executable program text (not writable)
Static:	static data
Heap:	dynamically allocated global memory (grows upward)
Stack:	local memory for function calls (grows downward)



2. The # Preprocessor

- The preprocessor runs before the compiler proper
  - modifies the source code
  - processes preprocessor instructions
    - lines beginning with #
  - strips out comments

Processor Instructions

- #include
  - insert a file
- #define
  - define a macro
- #ifdef, #ifndef, #endif
  - check to see if a macro has been defined

#include

- Inserts a file or header into the current source code.
- Two versions
  - #insert <headerName>
    - inserts a system header file from a location defined when the compiler was installed
  - #insert “fileName”
    - inserts a file from the current directory

Header Files

- C++ (along with C) uses *header files* as to hold definitions for the compiler to use while compiling.
- A source file (file.cpp) contains the code that is compiled into an object file (file.o).
- The header (file.h) is used to tell the compiler what to expect when it assembles the program in the linking stage from the object files.
- Source files and header files can refer to any number of other header files.

C++ language headers aren't referred to with the .h suffix. <iostream> provides definitions for I/O functions, including the *cout and cin* function.

```
#include <iostream>

using namespace std;

int main()
{
    string hello = "Hello";
    string world = "world!";
    string msg = hello + " " + world ;
    cout <<  msg << endl;
    msg[0] = 'h';
    cout <<  msg << endl;
    return 0;
}
```

#define

- Used to define macros (symbols that the preprocessor will later substitute for)
- Sometimes used to supply constants

```
#define VersionNumber "1.0Beta1"
int main()
{
cout << "Running version “<< VersionNumber<< endl;
}
```

- Much more elaborate macros are possible, including ones with parameters

Macro Example

```
#include <iostream>
using namespace std;

// Define a constant for the value of PI
#define PI 3.14159

int main()
{
    double radius = 4.0;


    double area = PI * radius * radius;

    cout << "Area of circle with radius " << radius << " is " << area;

    return 0;
}
```

Macro Example

```
#include <iostream>
using namespace std;
#define PRINT(x) cout << "Value is: " << x

int main()
{
    int value = 42;
    PRINT(value);
    return 0;
}
```

Macro Example

```
#include <iostream>
#define PRINT_RECTANGLE(width, height)
    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
            {
            std::cout << "*";
             }
         std::cout << std::endl;
    }

int main()
{

PRINT_RECTANGLE(4, 3);
    return 0;
}
```

3. Declarations and Definitions

- Some of the most common error messages are
  - … is undeclared
  - … is undefined
  - … is defined multiple times

- Fixing these requires that you understand the difference between declarations and definitions
  - and how they relate to the program structure

Declarations

- A declaration in C++
  - introduces a name for something
  - tells what “kind” of thing it is
  - gives programmers enough information to use it

Definitions

- A definition in C++
  - introduces or repeats a name for something
  - tells what “kind” of thing it is
  - tells what value it has and/or how it works
  - gives the compiler enough information to generate this and assign it an address

4. main() Function

main.cpp

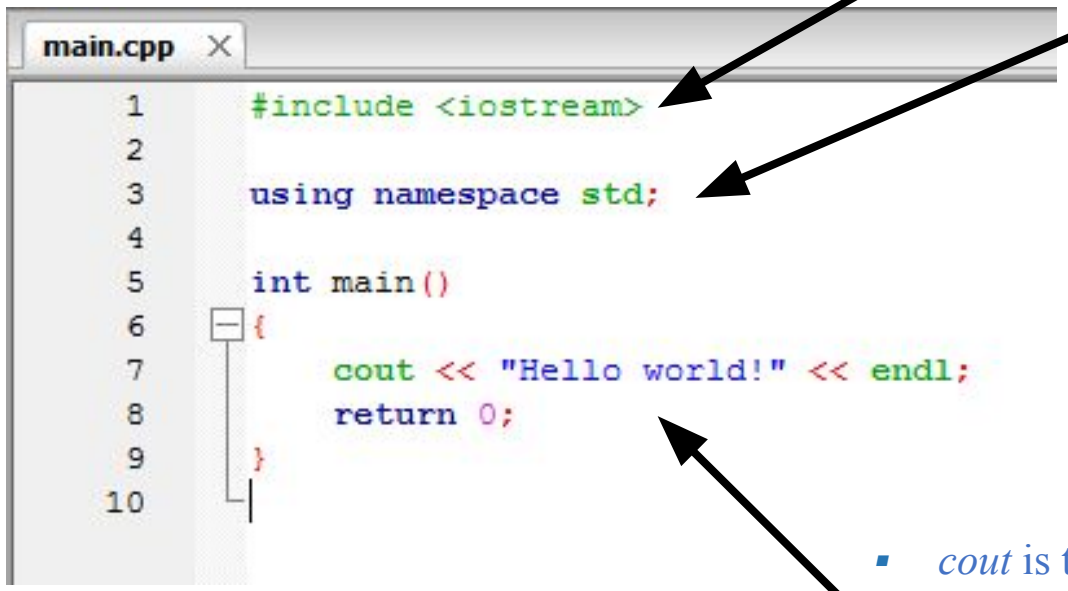
```
1   #include <iostream>
2
3   using namespace std;
4
5   int main()
6   {
7       cout << "Hello world!" << endl;
8       return 0;
9   }
10
```

The *main* routine – the start of **every** C++ program! It returns an integer value to the operating system and (in this case) takes no arguments: main()

The **return** statement returns an integer value to the operating system after completion. 0 means "no error". C++ programs **must** return an integer value.

loads a *header* file containing function and class definitions

Loads a *namespace* called *std*. Namespaces are used to separate sections of code for programmer convenience. To save typing we'll always use this line in this tutorial.



- *cout* is the *object* that writes to the stdout device, i.e. the console window.
- It is part of the C++ standard library.
- Without the "using namespace std;" line this would have been called as *std::cout*. It is defined in the *iostream* header file.
- << is the C++ *insertion operator*. It is used to pass characters from the right to the object on the left. *endl* is the C++ newline character.

Comments

- C++ syntax is very similar to C, Java, or C#.

- Curly braces are used to denote a code block (like the main() function):

{ … some code … }

- Statements end with a semicolon:

- Comments are marked for a single line with a // or for multilines with a pair of /* and */ :

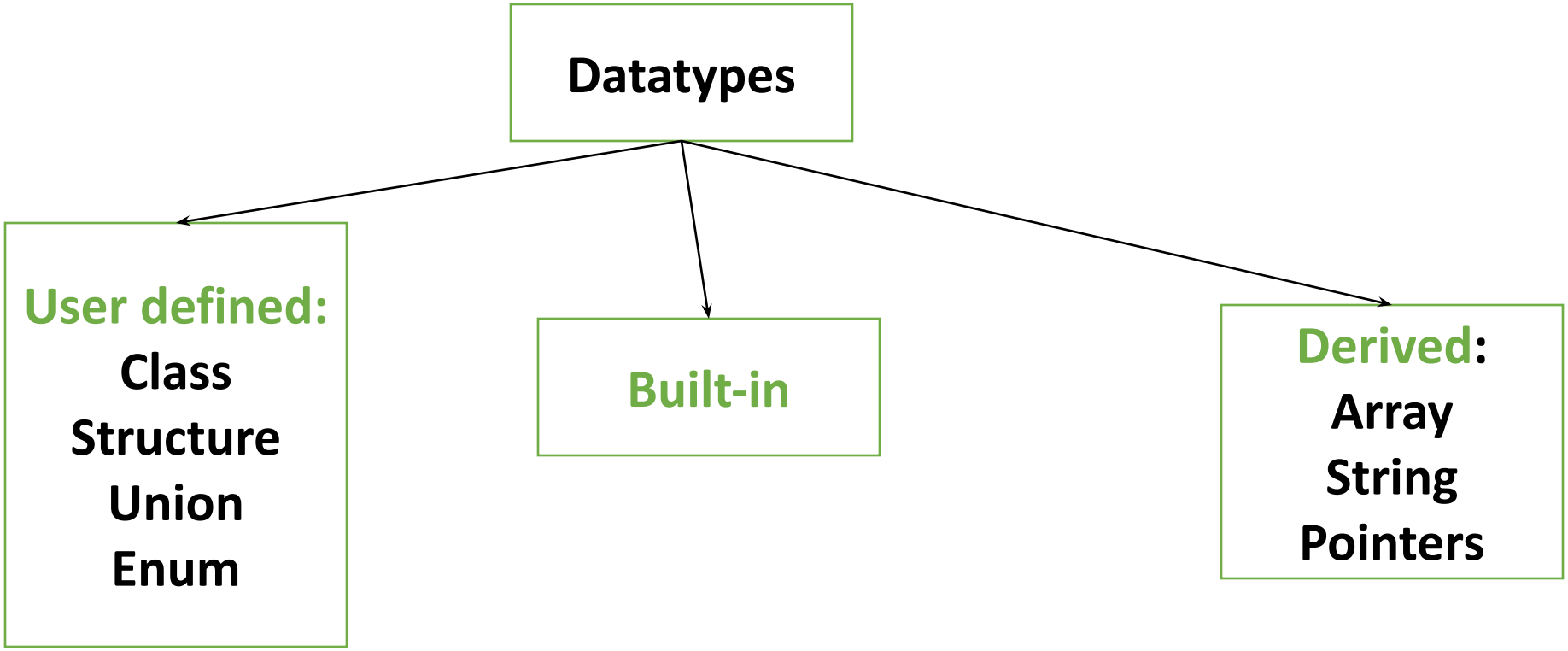
```
// this is a comment.
/* everything in here
        is a comment */
```

- Variables can be declared at any time in a code block.

Tokens, Keywords and identifiers

- Tokens , a smallest individual units.
  - Keywords, identifiers, constants,..etc
- Keywords implements specific c++ language features
- Identifiers refers the names of variables, functions, arrays..etc

Data types


Datatypes
User defined:
Class
Structure
Union
Enum
Built-in
Derived:
Array
String
Pointers

Structure

- Structure is the collection of variables of different types under a single name for better visualization of problem.
- Arrays is also collection of data but arrays can hold data of only one type whereas structure can hold data of one or more types.

```
struct person
{
 char name[50];
int age;
float salary;
 };
```

How to define a structure variable?

```
person bill;
```

How to access members of a structure?

```
bill.age = 50;
```

Struct Example

```
#include <iostream.h>
struct person
{
char name[50];
 int age;
float salary;
 };

 int main()
{
 person p1;
 cout << "Enter Full name: ";
 cin>>p1.name;
 cout << "Enter age: ";
 cin >> p1.age;
 cout << "Enter salary: ";
 cin >> p1.salary;
 cout << "\nDisplaying Information." << endl;
 cout << "Name: " << p1.name << endl;
 cout <<"Age: " << p1.age << endl;
 cout << "Salary: " << p1.salary;
 return 0;
 }
```

Union

- Unions in C++ is a user defined data type that uses the same memory as other objects from a list of objects. At an instance it contains only a single object.
- Example

```
#include <iostream.h>
union Emp
 {
 int num;
 double sal;
 };
 int main()
{
Emp value;
value.num = 2;
cout << "Employee Number::" << value.num
<< "\nSalary is:: " << value.sal << endl;
value.sal = 2000.0;
cout << "Employee Number::" << value.num
<< "\nSalary is:: " << value.sal << endl;
return 0;
}
```

Ouput

```
Employee number is::2
Salary is::0
Employee number is::0
Salary is::2000
```

Union

- Example

```
union test
{
    int x;
    char arr[8];
    int y;
};
int main()
{
    cout<< sizeof(union test) <<endl;
    return 0;
}
```

Union

- Example

```
union test
{
    int x;
    char arr[8];
    int y;
};
int main()
{
    cout<< sizeof(union test) <<endl;
    return 0;
}
```

enum

- Enum, which is also known as enumeration is a user-defined data type that consists of integral constants.
- It provides a way to define and group integral constants.
- To define an enumeration, keyword enum is used.
- Enums are generally used when you expect the variable to select one value from the possible set of values.

enum

```
#include <iostream>
using namespace std;
// Defining enum Year
enum year {Jan=0, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec};
int main()
{
    int i;
for (i = Jan; i <= Dec; i++)
        cout << i << " ";
    return 0;
}
```

Output

0 1 2 3 4 5 6 7 8 9 10 11

enum

```
#include <iostream>
using namespace std;
enum colors{red=5, black};
enum suit{heart, diamond=8, spade=3, club};
int main() {
   cout <<"The value of enum color : "<<red<<","<<black;
   cout <<"\nThe default value of enum suit :
"<<heart<<","<<diamond<<","<<spade<<","<<club;
   return 0;
}
```

Output

The value of enum color : 5,6
The default value of enum suit : 0,8,3,4

enum

```
#include <iostream>
    using namespace std;
    enum  cat
    {
        temp = 7, tempp, temppp;
    };
    int main()
    {
        float div = 18;
        div /= tempppp;
        cout << div << endl;
        return 0;
    }
```

Output

2

BUILT-IN Datatypes

Data Type	Size	Range
char	1 byte	-127 to 127 or 0 to 255
int	4 bytes	-2147483648 to 2147483647
unsigned int	4 bytes	0 to 4294967295
signed int	4 bytes	-2147483648 to 2147483647
short int	2 bytes	-32768 to 32767
long int	8 bytes	-2,147,483,648 to 2,147,483,647
long long int	16 bytes	-(2^63) to (2^63)-1
float	4 bytes	
double	8 bytes	
long double	16 bytes	
Bool	1 bytes	

Array

- Array is a collection of elements of same type referred using a unique name.
- Each element of an array are stored in memory locations indexed from 0 through n number of its elements.
- The lowest indexed will be the first element and highest indexed the last element of an array.
- type array_name[array_size_1]
- int age[5];

Example

```
#include <iostream.h>
  void main()
  {
  int i;
  float mark[6];
  cout << "Enter the marks of your 6 subjects:: \n";
  for(i=0; i<6; i++)
  {
  cin >> mark[i];
  }
  float sum=0;
for(i=0;i<6;i++)
  {
  sum += mark[i];
  }
  float ave = sum/6;
  cout << "Average Marks is::" << ave << '\n';
  }
```

Strings

- C++ provides convenient and powerful tools to manipulate strings.

- There are two types of strings commonly use in C++ programming language:

1. Strings that are objects of string class (The Standard C++ Library string class)

2. C-strings (C-style Strings)

Strings

- C++ provides convenient and powerful tools to manipulate strings.

- There are two types of strings commonly use in C++ programming language:

1. Strings that are objects of string class (The Standard C++ Library string class)

2. C-strings (C-style Strings)

C Style strings in C++

- char str[] = "C++";
- char str[4] = "C++";
- char str[] = {'C','+','+','\0'};
- char str[4] = {'C','+','+','\0'};

Example

```
#include <iostream>
#include<string>
using namespace std;
int main()
{
char str[100];
cout<<"Enter a string: ";
cin>>str;
cout<<"You entered: "<<str<<endl;
cout<<"\nEnter another string: ";
cin>>str;
cout<<"You entered: "<<str<<endl;
return 0;
}
```

Example

```
#include <iostream>
using namespace std;

int main()
{
  string S = “Hello World";
  cout << "Your string is= ";
  cout << S << endl;
  return 0;
}
```

C++ strings

Formatted Input: Stream extraction operator

cin >> stringObject

Unformatted Input: getline function for a string

getline(cin, s)
//reads an entire line of characters into s

string is not a pointer, but a class

Example

```
#include <iostream>
using namespace std;

int main()
{
  string s;
  cout<<"Enter String"<<endl;
  cin>>s;
  cout<<"String is: "<<s<<endl;
  return 0;
}
```

Example

```
#include <iostream>
using namespace std;
int main()
{
  string s;
  cout << "Enter String" << endl;
  getline(cin, s);
  cout << "String is: " << s << endl;
  return 0;
}
```

THANKS…..

RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering



B.Tech. 2nd Year

Object Oriented Methodologies (CS224)

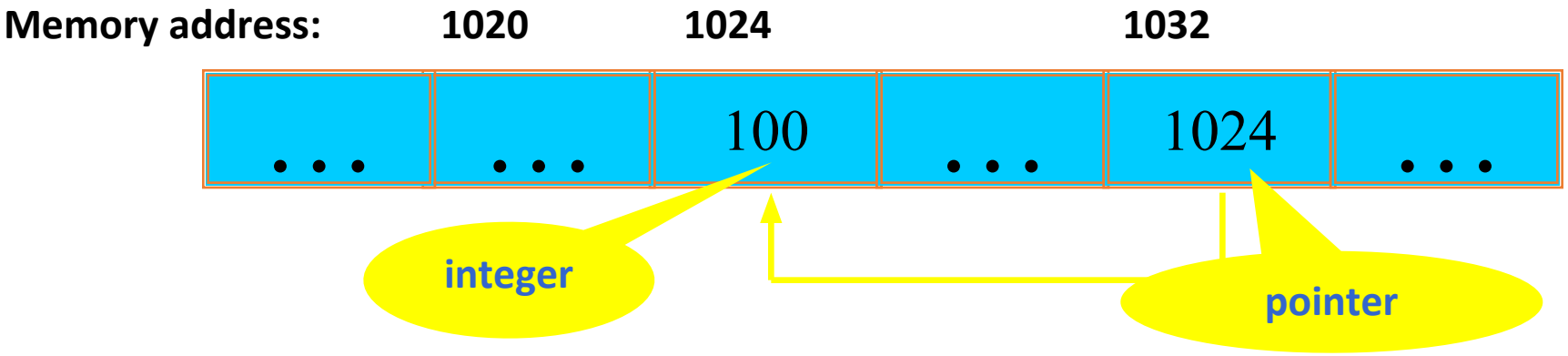
By

Dr. Kalka Dubey

Pointers

- A pointer is a variable used to store the address of a memory cell. We can use the pointer to reference this memory cell

datatype *var-name;



Operations

1. Define a pointer variables.

2. Assign the address of a variable to a pointer.

3. The value at the address available in the pointer variable and this is done by using unary operator * that returns the value of the variable located at the address specified by its operand.

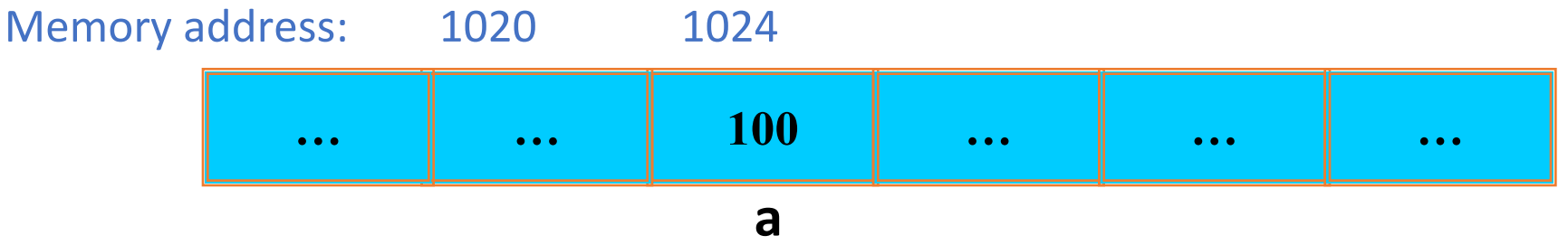
Pointers

- Store the address of other variable

```
int x;        // int variable
int *ip;     // int pointer
ip=&x;     //address of x assigned to ip
*ip=10;   // 10 assigned to x indirectly
```

Address operator &

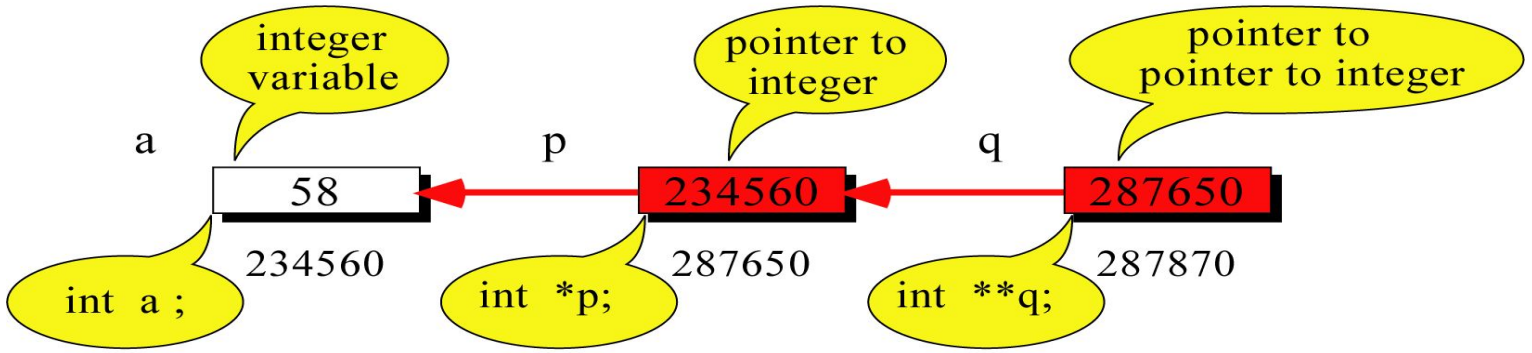
- *The "address of" operator* (&) gives the memory address of the variable




```
int a = 100;
//get the value,
cout << a;    //prints 100
//get the memory address
cout << &a;   //prints 1024
```

Pointer to pointer

```
// Local Declarations
int         a ;
int        *p ;
int       **q ;
```

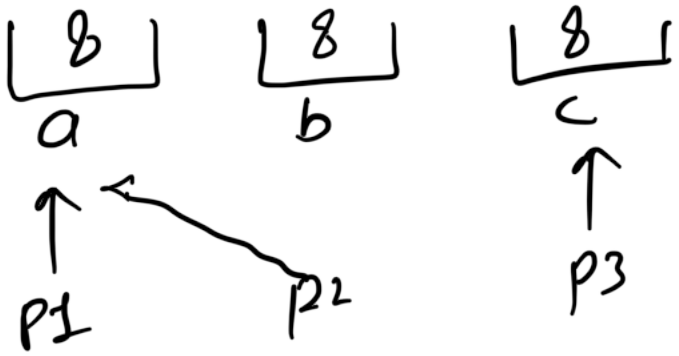


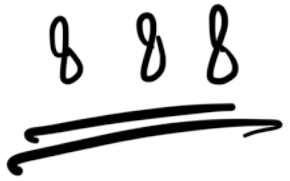

```
// Statements
a = 58 ;
p = &a ;
q = &p ;
cout <<       a << " ";
cout <<      *p << " ";
cout <<     **q << " ";
```

Example

```
int a = 100, b = 88, c = 8;
int *p1 = &a, *p2, *p3 = &c;
p2 = &b;          // p2 points to b
p2 = p1;     // p2 points to a
b = *p3;          //assign c to b
*p2 = *p3; //assign c to a
cout << a << b << c;
```

?





Null pointer

- A pointer that is assigned NULL is called a null pointer

```
int main ()
{
   int  *ptr = NULL;
   cout << "The value of ptr is " << ptr ;
    return 0;
}
```

To check for a null pointer you can use an if statement as follows:

- if(ptr) // succeeds if p is not null
- if(!ptr) // succeeds if p is null

Pointer arithmetic & using arrays

- You can perform arithmetic operations on a pointer just as you can a numeric value.
- There are four arithmetic operators that can be used on pointers: ++, --, +, and –
- For example, a pointer that points to the beginning of an array can access that array by using either pointer arithmetic or array-style indexing.

Example

```
int main ()
{
   int  var[3] = {10, 100, 200};
   int  *ptr;

   // let us have array address in pointer.
   ptr = var;
   for (int i = 0; i < 3; i++)
   {
      cout << *ptr << endl;

      // point to the next location
      ptr++;
   }
   return 0;
}
```

C++ array of pointers

- int *ptr[MAX];

- This declares pointer as an array of MAX integer pointers.

- Thus, each element in ptr, now holds a pointer to an integer value.

Exercise 1

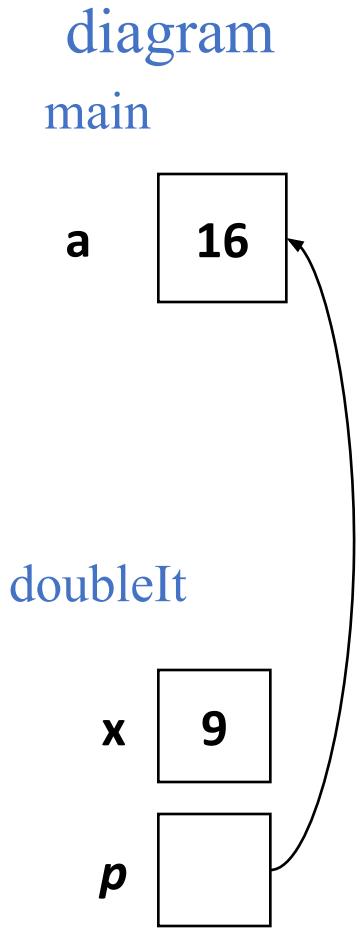
```
int main ()
{
   int  var[MAX] = {10, 100, 200};
   int *ptr[MAX];

   for (int i = 0; i < MAX; i++)
   {
      ptr[i] = &var[i]; // assign the address of integer.
   }
   for (int i = 0; i < MAX; i++)
   {
      cout << "Value of var[" << i << "] = ";
      cout << *ptr[i] << endl;
   }
   return 0;
}
```

Passing pointer to functions

The code

```
void doubleIt (int x, int * p)
{
   *p = 2 * x;
}
int main(int argc, const char * argv[])
{
   int a = 16;
   doubleIt(9, &a);
   cout<<a;
   return 0;
}
```



Exercise 2

```
#include <iostream>
using namespace std;

int main()
{
   const int i = 20;
   const int* const ptr = &i;
   (*ptr)++;
   int j = 15;
   ptr = &j;
   cout << i;
   return 0;
}
```

Error : You cannot Modify constant values.

Modifying const pointer

Output

Compile error

Exercise 3

```
#include <iostream>
using namespace std;
int main()
{
   int num[5];
   int* p;
   p = num;
   *p = 10;
   p++;
   *p = 20;
   p = &num[2];
   *p = 30;
   p = num + 3;
   *p = 40;
   p = num;
   *(p + 4) = 50;
   for (int i = 0; i < 5; i++)
      cout << num[i] << ", ";
   return 0;
}
```

Output

10, 20, 30, 40, 50,

Exercise 4

```
#include <iostream>
using namespace std;
int main()
{
   int arr[] = { 4, 5, 6, 7 };
   int* p = (arr + 1);
   cout << *arr + 10;
   return 0;
}
```

Output

14

Reference

- When a variable is declared as a reference, it becomes an alternative name for an existing variable.
- A variable can be declared as a reference by putting ‘&’ in the declaration.
- If a function receives a reference to a variable, it can modify the value of the variable.
- We can use references in for each loop to modify all elements.

Reference

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;

    // ref is a reference to x.
    int &ref = x;

    // Value of x is now changed to 20
    ref = 20;
    cout << "x = " << x << '\n';

    // Value of x is now changed to 30
```

Output

x = 20

ref = 30

Exercise 1

```
#include <iostream>
using namespace std;

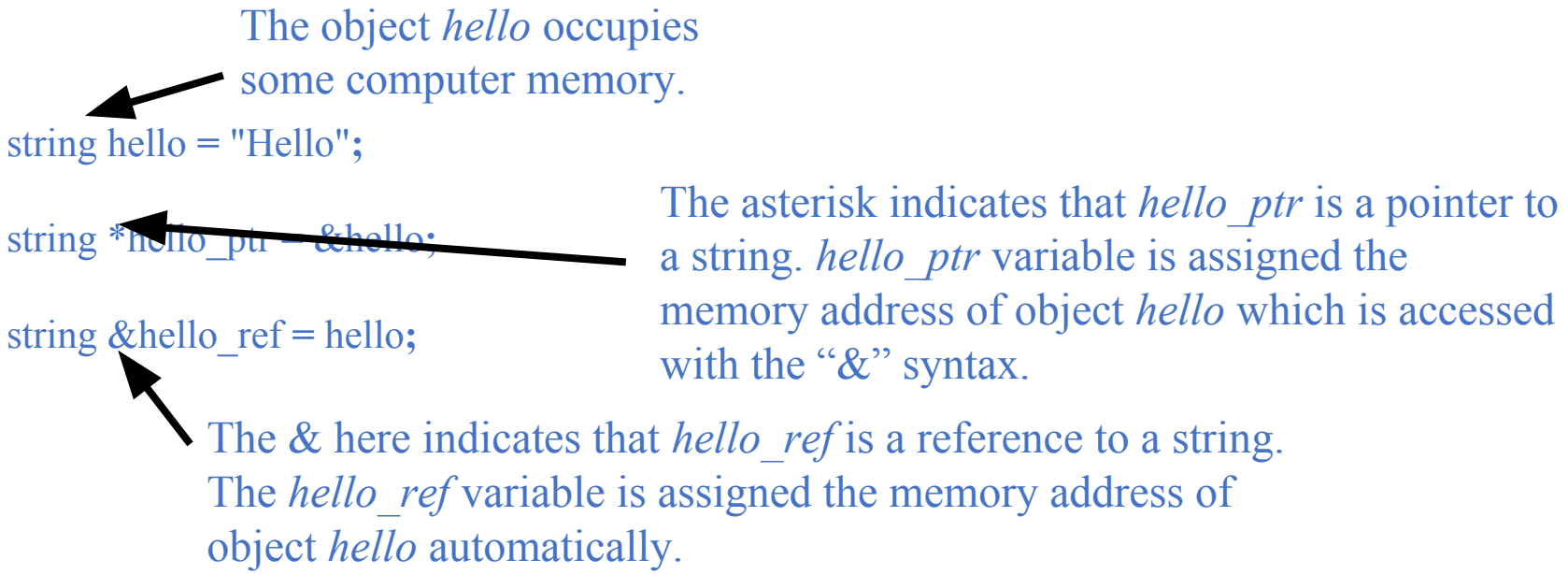
int main()
{
   int a = 32, *ptr = &a;
   char ch = 'A‘;
  char &cho = ch;

   cho += a;
   *ptr += ch;
   cout << a << ", " << ch << endl;
   return 0;
}
```

Output

129, a

Reference and Pointer Variables



- Variable and object values are stored in particular locations in the computer's memory.
- Reference and pointer variables **store the memory location of other variables.**
- Pointers are found in C. References are a C++ variation that makes pointers easier and safer to use.

Difference Between Pointer and Reference

↗ A diff. variable storing address

↳ Not having any existance, just a nickname.

- A pointer can be declared as void but a reference can never be void.
- The pointer variable has n-levels/multiple levels of indirection i.e. single-pointer, double-pointer, triple-pointer. Whereas, the reference variable has only one/single level of indirection.
- Reference variable cannot be updated.
- References are less powerful than pointers.
- Declaration of a Reference variable is preceded with the '&' symbol (but do not read it as "address of").
- References are safer and easier to use

- `p = &a;` ✓ `p = &b;` ✓ | `int &r = a;` `r = b;` ✗
- Pointers to NULL ✓ Reference to NULL ✗
- `p++, p--` ✓ `r++, r--` ✗

Input operators

cin >> variable-name;

Meaning: read the value of the variable called <variable-name> from the user

Example:

```
cin >> a;
cin >> b >> c;
cin >> x;
cin >> my-character;
```

Output operators

cout << variable-name;

Meaning: print the value of variable <variable-name> to the user

cout << “any message “;

Meaning: print the message within quotes to the user

cout << endl; Meaning: print a new line

Example:

```
cout << a;
cout << b << c;
cout << “This is my character: “ << my-character << “ end ” << endl;
```

Operators

Arithmetic operators

operators	met	Example
+	Addition	x=10,y=5 x+y -> 21
-	Subtraction	X-y -> 5
*	Multiplication	X*y -> 50
/	Division	x/y -> 2
%	Modulo Division	X%y -> 0

Relational Operators

Operator	Meaning	Example	Result	value
<	Less Than	x=10,y=5 X<y	False	0
>	Greater Than	X>y	True	1
<=	Less than or Equal to	X<=y	False	0
>=	Greater than or equal to	X>=y	True	1
!=	Not Equal to	X!=y	False	0
==	Equal To	X==y	False	0

Operators

Logical Operators

operators	met	Example
&&	Called Logical AND operator. If both the operands are non-zero, then condition becomes true.	Assume variable A holds 1 and variable B holds 0 (A && B) is false
||	Called Logical OR Operator. If any of the two operands is non-zero, then condition becomes true.	(A || B) is true
!	Multiplication	!(A && B) is true

Assignment Operators

Operator	Meaning	Example
=	Simple assignment operator	C = A + B
+=	Add AND assignment operator	C += A
-=	Add AND assignment operator	C -= A
*=	Multiply AND assignment operator	C *= A
/=	Divide AND assignment operator	C /= A
%=	Divide AND assignment operator	C %= A

Others Operators

- Scope resolution operator ::
- Member dereferencing operator ->, *, ::*
- Memory management operator malloc(), sizeof(), new, delete
- Type cast operators type_name (expression)

Scope resolution operator → used to access Global Variable.

```
#include <iostream>
using namespace std;
int num = 50;
int main ()
{
int num = 100;

cout << " The value of the local variable num: " << num;
cout << "\n The value of the global variable num: " << ::num;
return 0;
}
```

Output

The value of the local variable num: 100

The value of the global variable num: 50

Member dereferencing operator

- **Pointer to a member declarator (::*) :** To declare a pointer to a member of a class.

- **Pointer to member operator(.*) :** To access a member using an object name and a pointer to that member.

- **Pointer to member operator (->*) :** To access a member using a pointer to the object and a pointer to that member.

Dynamic variable using new

```
#include <iostream>
#include <memory>
using namespace std;
int main()
{
int* ptr;
ptr = new int;
*ptr = 10;
  cout << "Address: " << ptr << endl;
  cout << "Value: " << *ptr;
    return 0;
}
```

Dynamic variable using new

```
#include <iostream>
using namespace std;
int main()
{
    int* arr = new int[5];
  cout << "Enter 5 values in the array: ";
    for(int i = 0; i < 5; i++)
  {
    cin >> arr[i];
  }
  cout << "\nArray elements: ";
  for(int i = 0; i < 5; i++)
  {
    cout << arr[i] << " ";
  }
  delete [] arr;
    for(int i = 0; i < 5; i++)
    {
      cout << arr[i] << " ";
    }

    return 0;
}
```

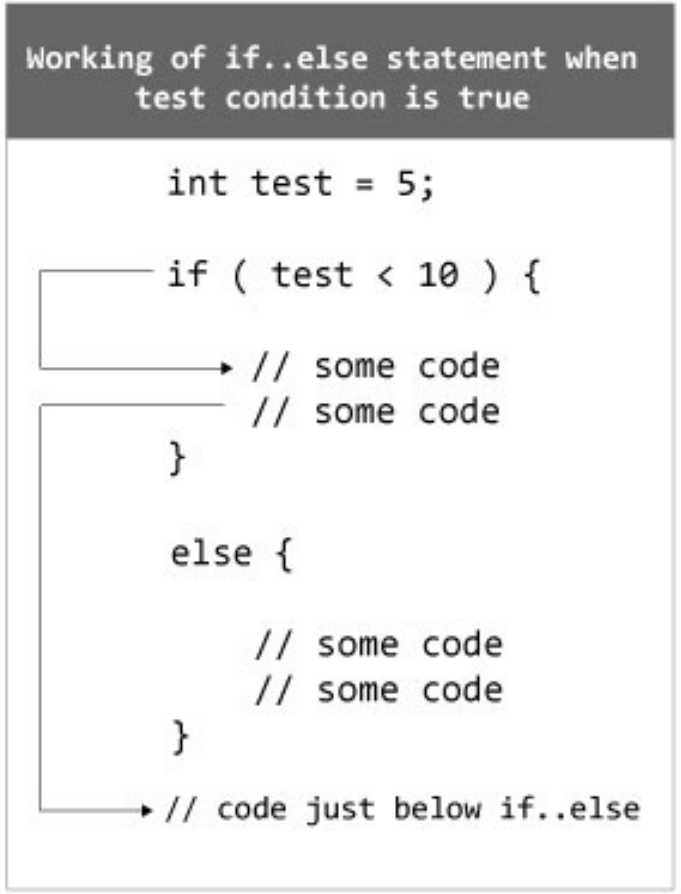
Type Casting

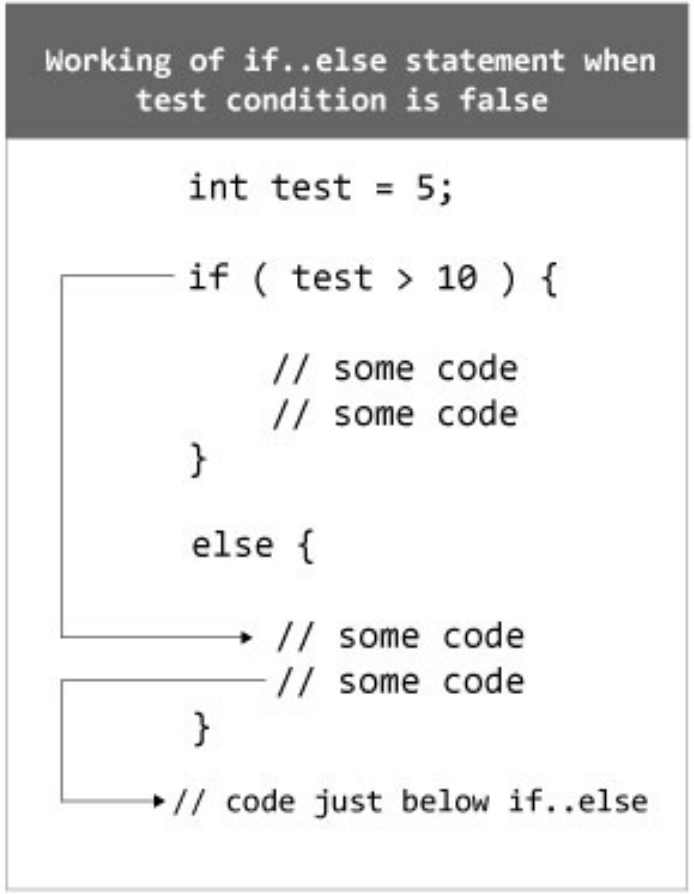
Used to convert one datatype into another data type.

```
#include<iostream.h>
#include<conio.h>
void main()
{
int a=10, b;
float c=2.3;
clrscr();
b=int (a+c);
Cout<<"The value of b is <<b;
getch();
}
```

Control structures

- if
  - if else
  - Nested if
- Switch
- do while
- While
- for





Loops

- For loop

```
for(initialization statement; test expression; update statement)
{
codes to be executed;
 }
```

- While loop

```
while (test expression)
{
statement/s to be executed.
 }
```

Loops

- Do while

```
do
 {
statement/s;
 } while (test expression);
```

Loops Example

- Write a C++ Program for Fibonacci Series.

Loops Example

```
#include<iostream>
using namespace std;
int main()
{
    int a=0,b=1,t,j=1,num;
    cout<<"enter the limit ";
    cin>>num;
    cout<<"The Fb Series is "<<a<<b;
    do
    {
        t=a+b;
        cout<<t;
        a=b;
        b=t;
```

Loops Example

Output

enter the limit 3

The Fb Series is 01123.

Loops Example

- Write a program to print all Armstrong numbers between 1 and 500.

Loops Example

```
// If sum of cubes of each digit of the number is equal to the number itself, then the number is called an Armstrong number.
#include<iostream>
using namespace std;

int main()
{
	int n,digit1,digit2,digit3;

	for(int i=1;i<=500;i++)
	{
		digit1=i/100;
		digit2=i/10 - digit1*10;
		digit3=i%10;

		if (digit1*digit1*digit1 + digit2*digit2*digit2 + digit3*digit3*digit3 == i)
			cout<<i<<endl;
	}


	return 0;
}
```

Loops Example

Output

1
153
370
371
407.

Loops Example

- Write a program to calculate HCF of Two given number.

Loops Example

```
#include<iostream>
using namespace std;
int main()
{
	int dividend, divisor, rem=1, hcf;
	cout<<"Enter two numbers : ";
	cin>>dividend>>divisor;
	while(rem!=0)
	{
		rem=dividend % divisor;
		if(rem==0)
			hcf=divisor;
		else
		{
			dividend=divisor;
			divisor=rem;
		}
	}
	cout<<"HCF is : "<<hcf;
	return 0;
}
```

Loops Example

Output

Enter two numbers : 30 105

HCF is : 15

Switch

```
switch (n) {
case constant1:
code/s to be executed if n equals to constant1;
break;
case constant2:
code/s to be executed if n equals to constant2;
break; . .
.
default:
code/s to be executed if n doesn't match to any cases;
 }
```

Functions

The return type is *float*.

The function arguments L and W are sent as type *float*.

```
float RectangleArea1(float L, float W) {
   return L*W ;
}


float RectangleArea2(const float L, const float W) {
   // L=2.0 ;
   return L*W ;
}

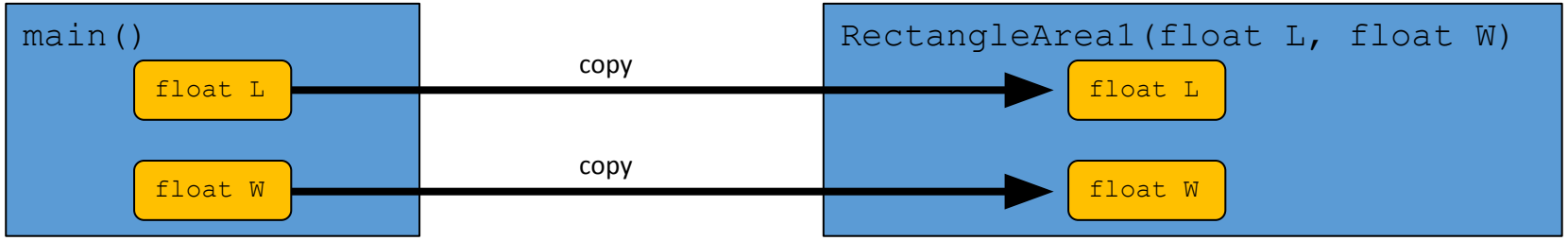

float RectangleArea3(const float& L, const float& W) {
   return L*W ;
}


void RectangleArea4(const float& L, const float& W, float& area) {
   area= L*W ;
}
```

Product is computed

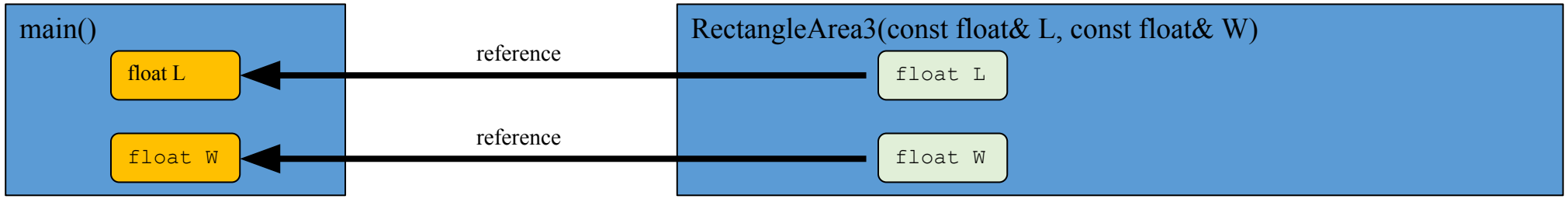


Pass by Value



- C++ defaults to *pass by value* behavior when calling a function.
- The function arguments are **copied** when used in the function.
- Changing the value of L or W in the RectangleArea1 function does **not** effect their original values in the main() function
- When passing objects as function arguments it is important to be aware that potentially large data structures are automatically copied!

Pass by Reference



- *Pass by reference* behavior is triggered when the & character is used to modify the type of the argument.
- This is the type of behavior you see in Fortran, Matlab, Python, and others.
- Pass by reference function arguments are **NOT** copied. Instead the compiler sends a *pointer* to the function that references the memory location of the original variable. The syntax of using the argument in the function does not change.
- Pass by reference arguments almost always act just like a pass by value argument when writing code except that changing their value changes the value of the original variable!!
- The *const* modifier can be used to prevent changes to the original variable in main().

Function Example

- Write a function that receives two numbers as an argument and display all prime numbers between these two numbers. Call this function from main( ).

Function Example

```
#include<iostream>
using namespace std;

void showprime(int, int);

int main()
{
	int x,y;
	cout<<"Enter first  number : ";
	cin>>x;
	cout<<"Enter second  number : ";
	cin>>y;
	showprime(x,y);
	return 0;
}
void showprime(int a, int b)
{
	bool flag;
	for(int i=a+1;i<=b;i++)
	{
```

Function Example

Output

```
Enter first number : 15
Enter second number : 70
17
19
23
29
31
37
41
43
47
53
59
61
67
```

Function Example

- Write a function called zero_small() that has two integer arguments being passed by reference and sets the smaller of the two numbers to 0. Write the main program to access the function.

Function Example

```
#include<iostream>
using namespace std;
void zero_small(int &,int &);
int main()
{
    int x,y;
    cout<<"Enter first number : ";
    cin>>x;
    cout<<"Enter second number : ";
    cin>>y;
    zero_small(x,y);
    cout<<"First number is : "<<x;
cout<<"\nSecond number is : "<<y;
```

Function Example

Output

Enter first number : 23

Enter second number : 6

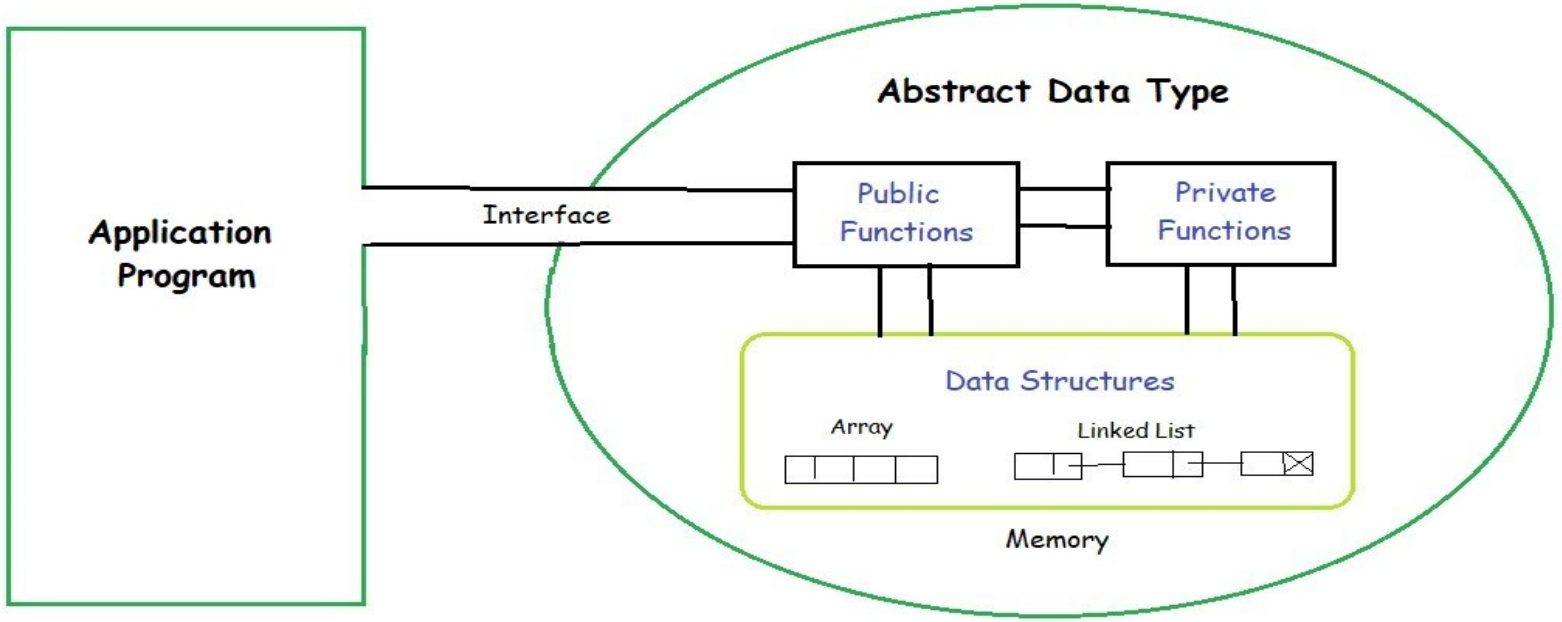
First number is : 23

Second number is : 0

Abstract Data Types

- Abstract Data type (ADT) is a type (or class) for objects whose behavior is defined by a set of values and a set of operations.
- The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented.
- It is called “abstract” because it gives an implementation-independent view.
- The process of providing only the essentials and hiding the details is known as abstraction.

Abstract Data Types



Abstract Data Types in C++

- Data abstraction allows a program to ignore the details of how a data type is represented.
- Abstraction (derived from a Latin word abs, meaning away from and traverse, meaning to draw) refers to the act of representing essential features without including the background details or explanations.
- C++ classes use the technique of abstraction and are defined as a list of abstract attributes such as width, cost, size etc and functions to operate on these attributes.
- They put in a nutshell all the essential properties of an object that are required to be created.
- The attributes are therefore called data members as they hold information.
- The functions that operate on those data members are termed member functions.
- Since C++ classes use the data abstraction concept, they are also termed abstract data types.

THANKS…..